Adaptive Green AI

Carbon-aware LLM orchestration

Routing, deferral, retrieval, safety, and an agentic coding harness — every request scored on the carbon it will actually cost.

HLD · LLD · Workflow · Problem · Solution · Features

Himanshu Tripathi · Hewlett Packard Enterprise · 14 July 2026

1 · Solution brief

Adaptive Green AI is a control plane that sits in front of a fleet of vLLM backends and turns every inference request into a sustainability-optimised routing decision. It is not a dashboard bolted onto an LLM stack: carbon is an input to the decision, not a number reported after the fact. A single FastAPI service profiles the prompt, scores every candidate model against carbon, latency, accuracy and cost, defers work that the grid cannot afford right now, dispatches to the greenest model that can actually do the job, and learns from the outcome — writing an HMAC-signed audit row for every decision so that any of it can be replayed or challenged.

> "The greenest model that can do the job" is the entire thesis. Both halves of that sentence carry weight — and the second half is the one everybody gets wrong.

The system has two distinct carbon levers, and they operate on different axes. The first is **routing in model space**: a 1.1B model answers a greeting for a fraction of the carbon of a 7B model, so the Composite Sustainability Score (CSS) ranks candidates and picks the smallest one that clears the accuracy floor and the SLA. The second is **routing in time**: grid carbon intensity swings by hundreds of gCO_2/kWh over a day, so work that nobody is waiting for is queued and dispatched into the cleanest window the 48-hour forecast offers. Identical joules, materially less carbon.

A third mechanism, added last, governs **agentic** work — where the request is not one forward pass but a loop of them. There the per-request logic inverts, and getting it wrong is expensive. Section 6 is about that inversion, and it is the most interesting engineering in the system.

What is in the box

Layer	Module	What it decides
Safety	nemo_guardrails.py	Blocks jailbreaks, injection and harmful content on the way in; PII, credentials and unsafe echoes on the way out. Pattern rails — no external LLM, so the safety layer costs no carbon of its own.
Grounding	advanced_rag.py	Hybrid dense + sparse retrieval, reciprocal-rank fusion, cross-encoder rerank. Answers get evidence; the evidence-sufficiency check decides whether the model is allowed to answer at all.
Understanding	routing_policies.py	Semantic profile: intent, complexity, SLA, accuracy floor, STEM domain, and modality (text / vision / image-gen).
Decision	routing_policies.py	CSS ranks every candidate on carbon, latency, accuracy and cost with RL-learned per-tier weights. The winner is the greenest feasible candidate.
Correction	quality_latency_estimator.py	Learns per-prompt accuracy and latency corrections online, feeding CSS. Carbon is never adjusted — the physics is not up for negotiation.
Timing	deferred_queue.py	Above the carbon threshold, work is queued and dispatched in the greenest forecast window inside its deadline.
Accounting	model_zoo.py	LLMCarbon operational + embodied carbon per candidate, per request. Manufacturing carbon is amortised, not ignored.
Learning	rl_controller.py	Online REINFORCE with an EMA baseline; every outcome nudges the tier's CSS weights. No offline training step.
Multimodal	multimodal.py	Vision analysis and carbon-capped image generation via pluggable NIM endpoints, with graceful fallback when none is configured.
Agentic	coding_agent.py	LangGraph harness that optimises carbon per successful completion — verifier-gated escalation, frozen spec, deferrable to a clean grid.
Evidence	decision_engine.py	Every decision becomes one HMAC-signed JSONL row. Every dashboard reads that file and nothing else.

2 · The problem

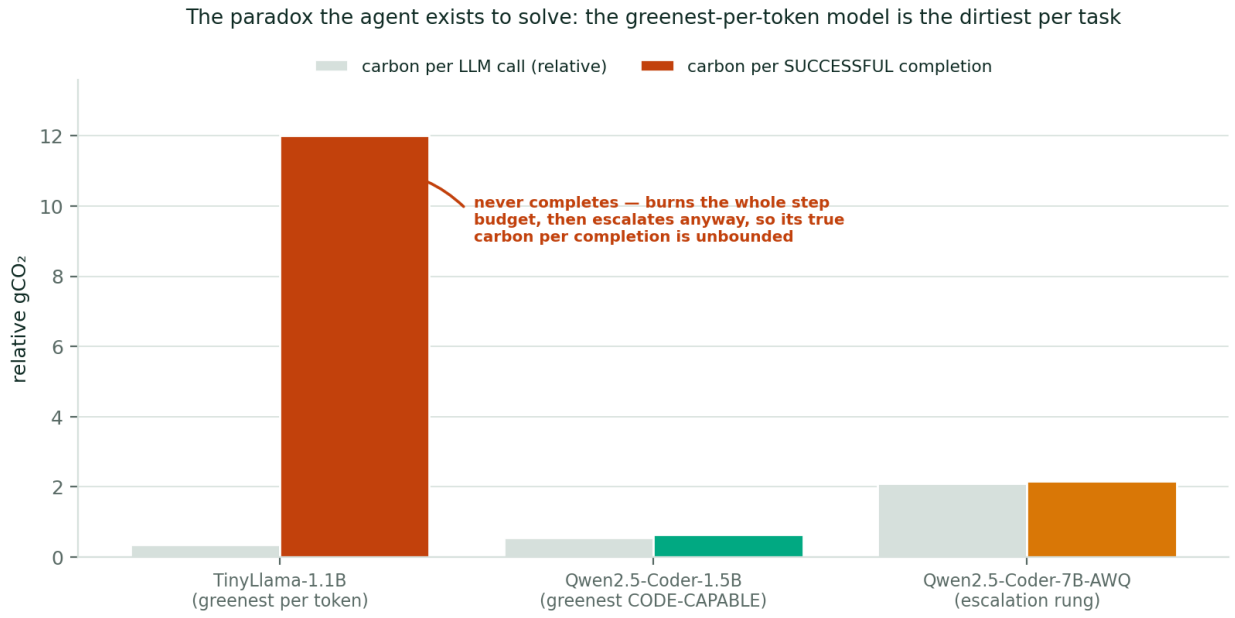
Inference, not training, is now where the carbon is. A model is trained once and then serves requests for years, and the aggregate of those requests dwarfs the one-off training run. Yet almost every serving stack makes the same decision for every request: send it to the biggest model available, because that is the safest thing to do for quality. The result is that a greeting, a lookup, and a hard analytical question all cost the same — and they all cost the maximum.

2.1 Three failures of the naïve stack

Failure	What it looks like in production	Cost
One model for every prompt	A 7B model answers “hi”. Capacity is provisioned for the hardest request and then spent on the easiest one.	Carbon scales with the worst case, not the average case.
Carbon is measured, not used	A dashboard reports gCO_2 after the fact. Nothing in the request path ever reads that number, so nothing changes.	Reporting without control. The metric moves only when humans intervene.
Time is ignored entirely	A batch job runs at 18:00 on a coal-heavy grid because that is when the cron fired, not because it had to.	The same work emits 2–3× more carbon than it would have six hours later.

2.2 And one failure that only appears when the model is an agent

The obvious fix — “always use the greenest model” — is right for a single request and catastrophically wrong for an agent. CSS scores carbon **per request**. An agent is a loop: its cost is tokens × steps × attempts. A model that is 3× cheaper per call but cannot finish the task does not save 3× the carbon; it burns the entire step budget, produces nothing, and then the system escalates to the big model anyway and pays for that too.

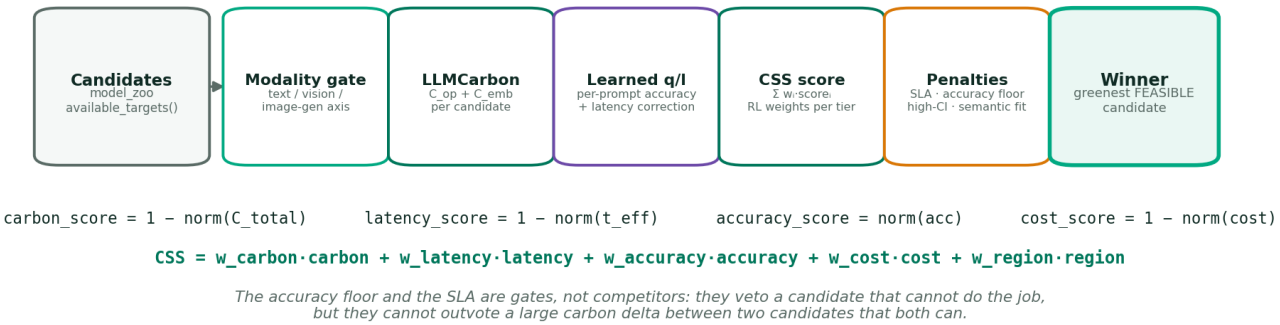


The greenest-per-token model is the dirtiest per completed task. TinyLlama cannot write working code at all, so its true carbon-per-completion is unbounded — it spends the budget and delivers nothing. The ladder therefore starts at the greenest *code-capable* rung.

This is not a hypothetical. It is the same trap that produced a real mis-route earlier in this project's history, when a coding prompt was sent to TinyLlama because TinyLlama was the greenest candidate on paper. The lesson generalises: **“feasible” must be judged against task completion, not against one forward pass.**

3 · The idea

Make carbon a first-class term in the objective function of the router, give the router a real choice of models, and then let it learn which trade-offs actually paid off.



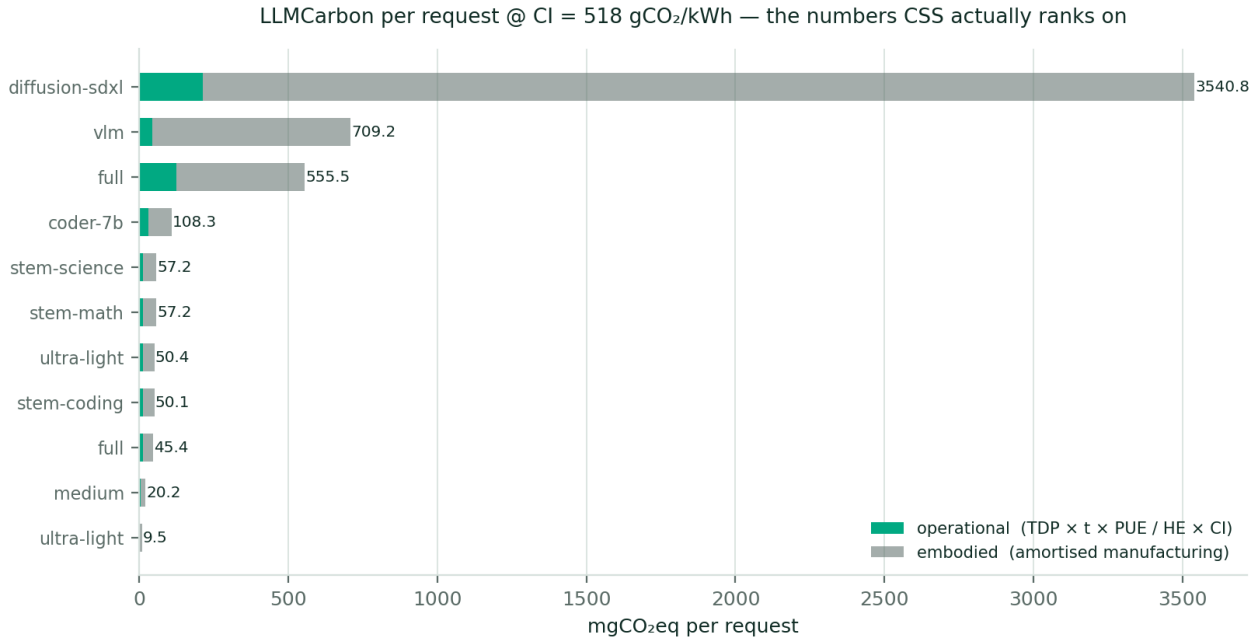
Four candidates enter, one decision leaves. The accuracy floor and the SLA are *gates* — they veto candidates that cannot do the job — but they cannot outvote a large carbon delta between two candidates that both can.

3.1 The Composite Sustainability Score

$$CSS = w_{carbon} \cdot carbon + w_{latency} \cdot latency + w_{accuracy} \cdot accuracy + w_{cost} \cdot cost + w_{region} \cdot region$$

Each dimension is min-max normalised across the live candidate set, so the score is a genuine comparison rather than an absolute. Carbon comes from the LLMCarbon model — operational carbon from the actual TDP, PUE and hardware efficiency of the device, plus embodied manufacturing carbon amortised over the device's lifetime inference volume. Both terms are stored on every candidate, so the audit log can prove the breakdown rather than assert a total.

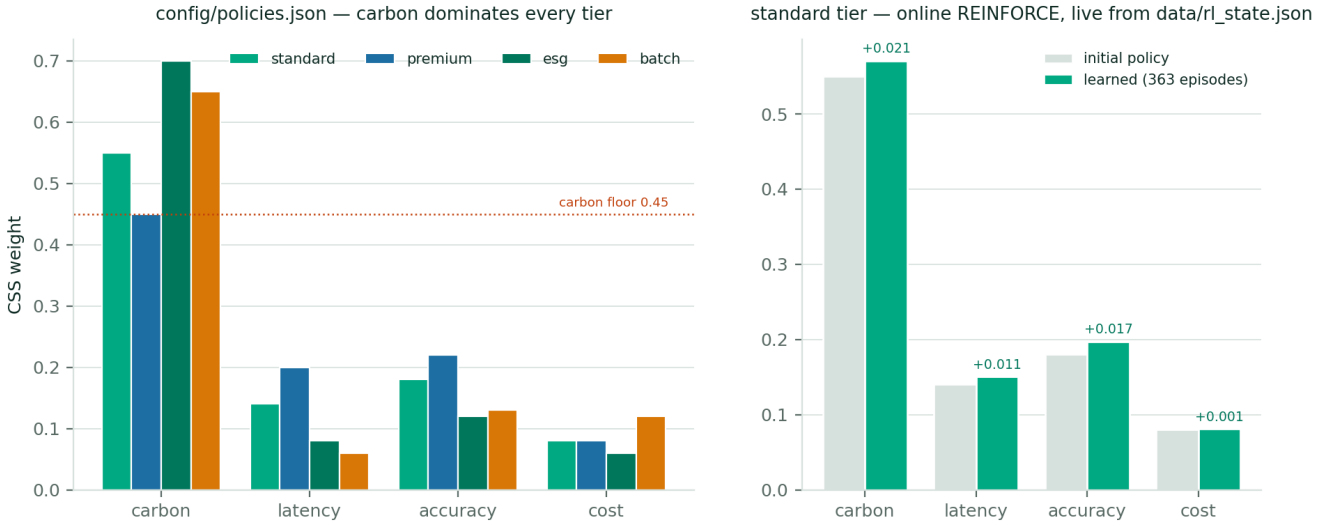
```
C_op = (TDP × t × PUE) / (HE_eff × 3.6e6) × CI × region_mult
C_emb = (mfg_kg × 1000) / (lifetime_y × annual_vol × avg_s) × t × device_share
C_total = C_op + C_emb
```



Operational and embodied carbon per request for every live candidate, computed with the formula above at the grid intensity actually measured on this deployment. These are the numbers CSS ranks on — not parameter counts, not vibes.

3.2 Carbon must dominate, or the system is just a router

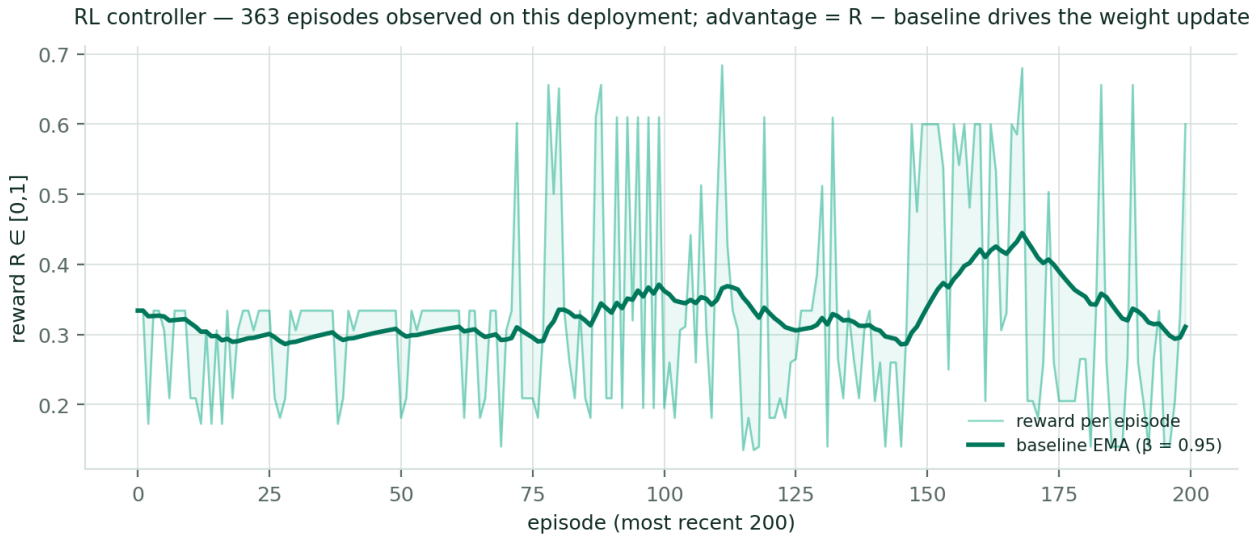
A sustainability platform whose weights let latency outvote carbon is a latency platform with a carbon dashboard. So the per-tier carbon weight is floored at 0.45 and reaches 0.70 for the ESG tier. Latency and accuracy retain enough influence to enforce the SLA and the accuracy floor through their penalty terms, but they cannot overturn a sizeable carbon delta on their own.



Left: the configured starting policy per tenant tier. Right: what online REINFORCE has actually learned on the standard tier after the episodes observed on this box — carbon weight drifted *up*, because the greener choice kept being rewarded.

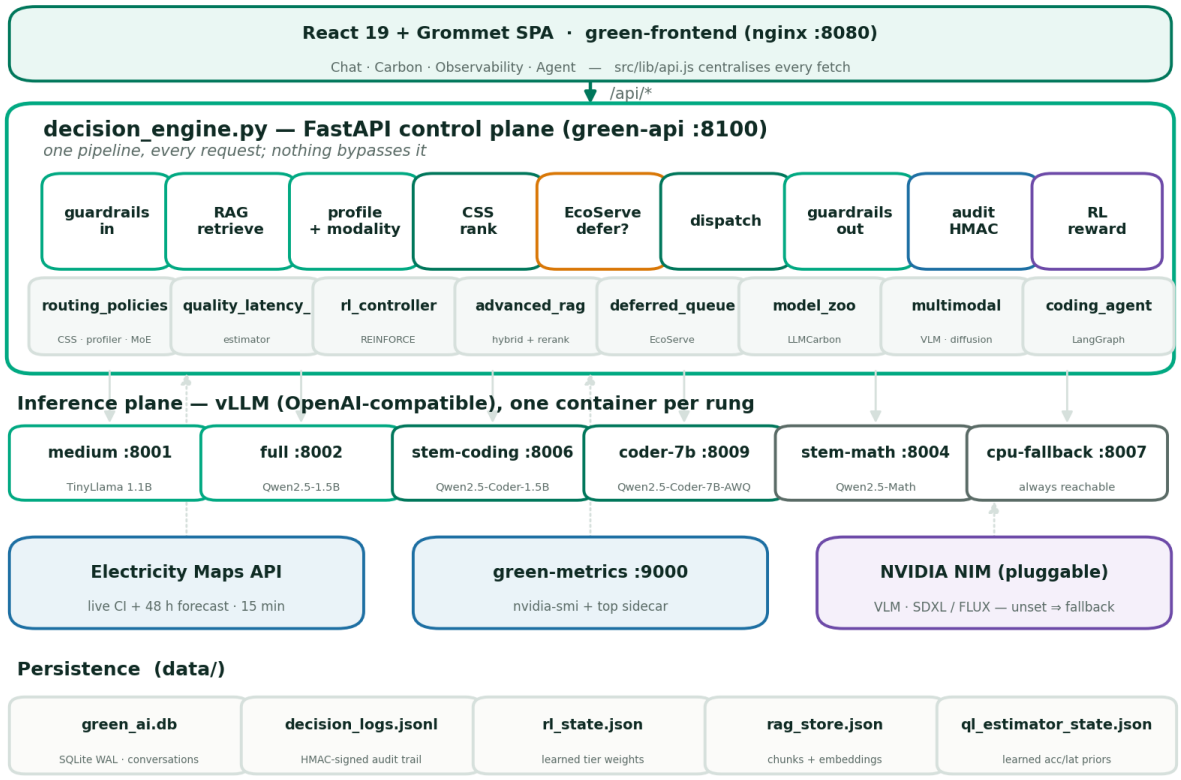
3.3 Learning, not tuning

The weights are a starting point, not a setting. Every completed request produces a reward — an SLA term, a carbon term, an accuracy-outcome term and a cost term — and the controller takes a REINFORCE step against an EMA baseline, projecting the result back onto the simplex with a floor so no dimension can collapse to zero. There is no offline training job, no nightly batch, and no slider in the UI. The policy is the accumulated consequence of every decision the system has made.



Real reward history from data/rl_state.json on this deployment. The advantage (R – baseline) is what actually moves the weights; the EMA is what makes a good decision in a bad hour still read as a good decision.

4 · High-level design



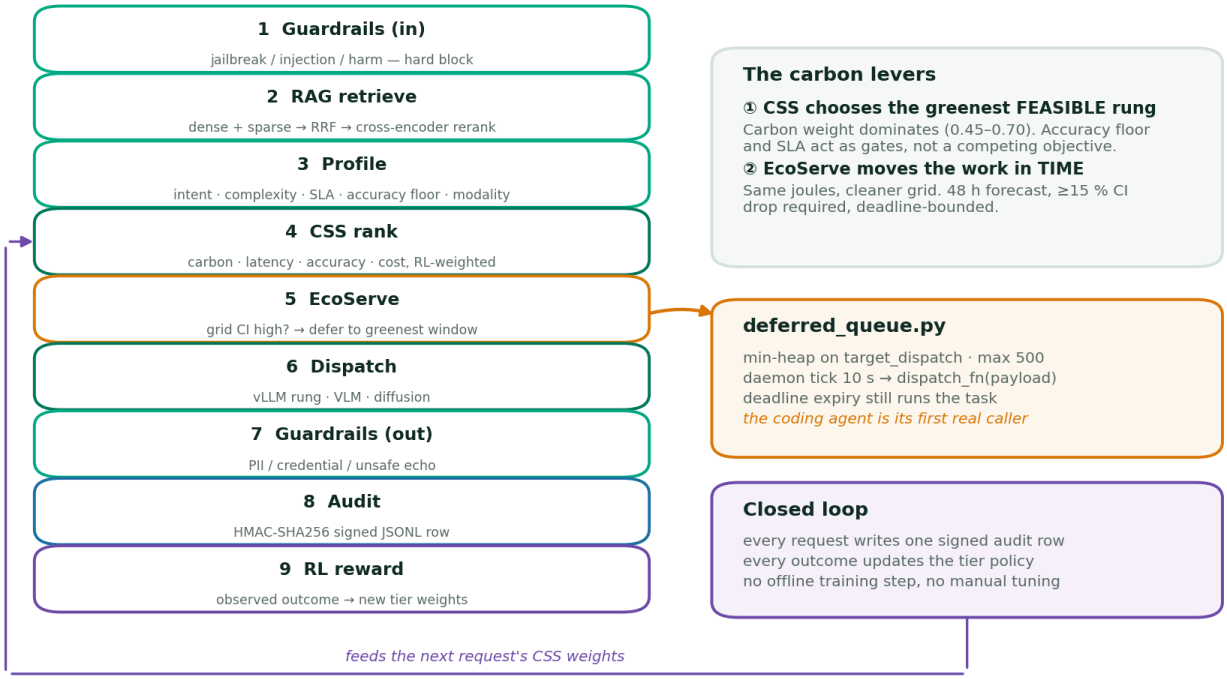
Every number the dashboards show is derived from decision_logs.jsonl — there is no second source of truth.

Every request enters through one pipeline. Nothing bypasses it — the failure paths are deterministic substitutes, not escape hatches.

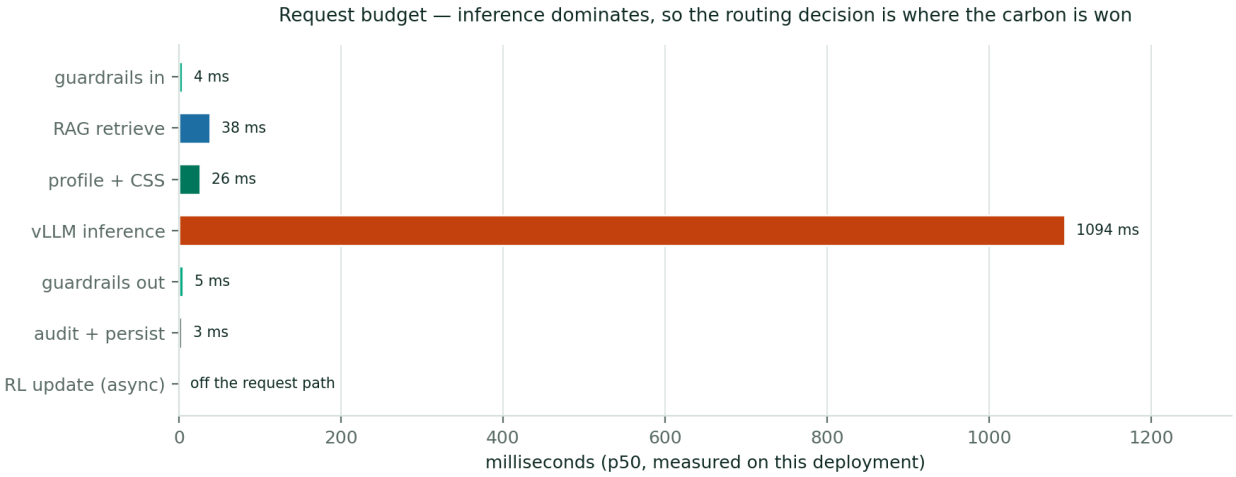
The control plane is a single FastAPI process. That is a deliberate choice: the routing decision needs the prompt profile, the grid signal, the GPU telemetry, the RL policy and the model registry all in the same place at the same moment, and distributing that state across services would buy nothing but latency and skew. The things that genuinely must not block the event loop — the `nvidia-smi` subprocess, the RL update, the deferred dispatch loop — are the things that were moved out.

Container	Port	Role
green-api	8100	Decision engine; the whole pipeline and every endpoint.
green-metrics	9000	Host telemetry sidecar. Isolated because system_metrics.sh can take >1 s and must never stall the API's event loop.
green-frontend	8080	React 19 + Grommet SPA behind nginx, which also reverse-proxies /api.
vllm-medium	8001	TinyLlama 1.1B — the cheap rung for easy prompts.
vllm-full	8002	Qwen2.5-1.5B — the general-purpose rung.
vllm-stem-coding	8006	Qwen2.5-Coder-1.5B — the greenest *code-capable* model, and the agent's first rung.
vllm-coder-7b	8009	Qwen2.5-Coder-7B-AWQ — the escalation rung, entered only on verifier evidence.
vllm-fallback	8007	CPU. Slow, but always reachable, so a GPU outage degrades rather than fails.

5 · Workflow — the life of a request



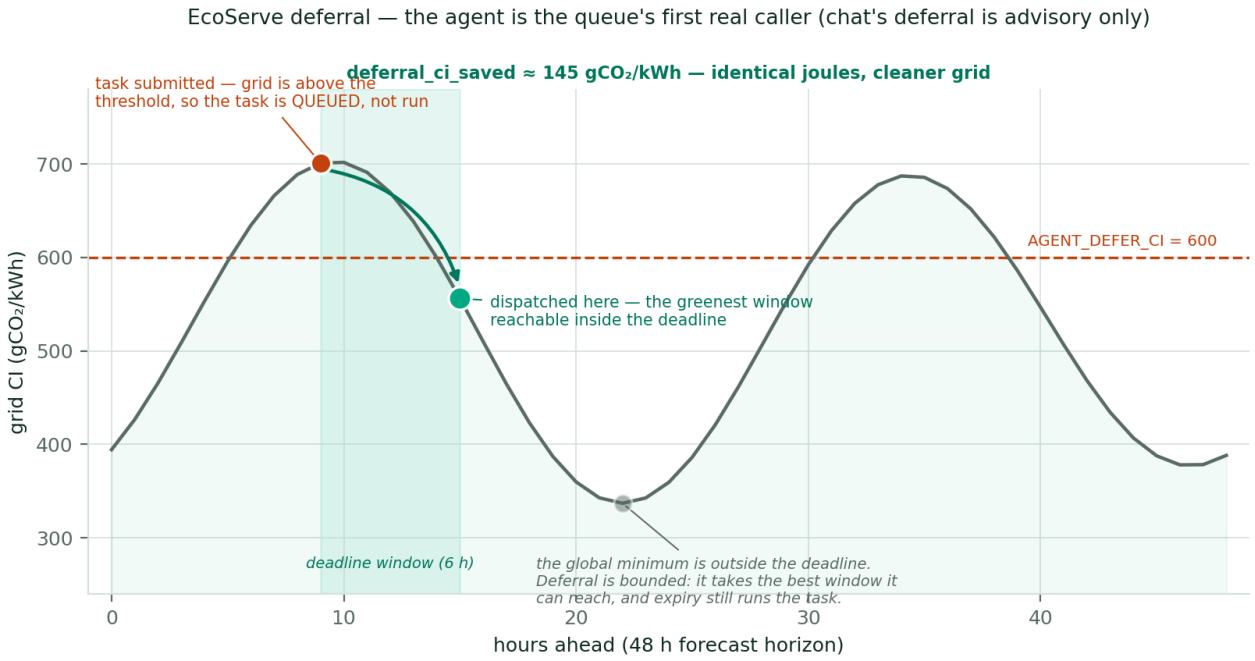
The pipeline, and the two branches that make it green: CSS moves work in *model space*, EcoServe moves it in *time*.



Where the milliseconds go. Inference dominates by two orders of magnitude — which is precisely why the decision about *which* model to invoke is where the carbon is won or lost, and why spending 26 ms to make that decision well is free.

The RL update runs on a background thread after the response has been returned. This matters more than it looks: learning is not on the critical path, so the system can afford to learn from every single request rather than sampling.

5.1 EcoServe — moving work in time



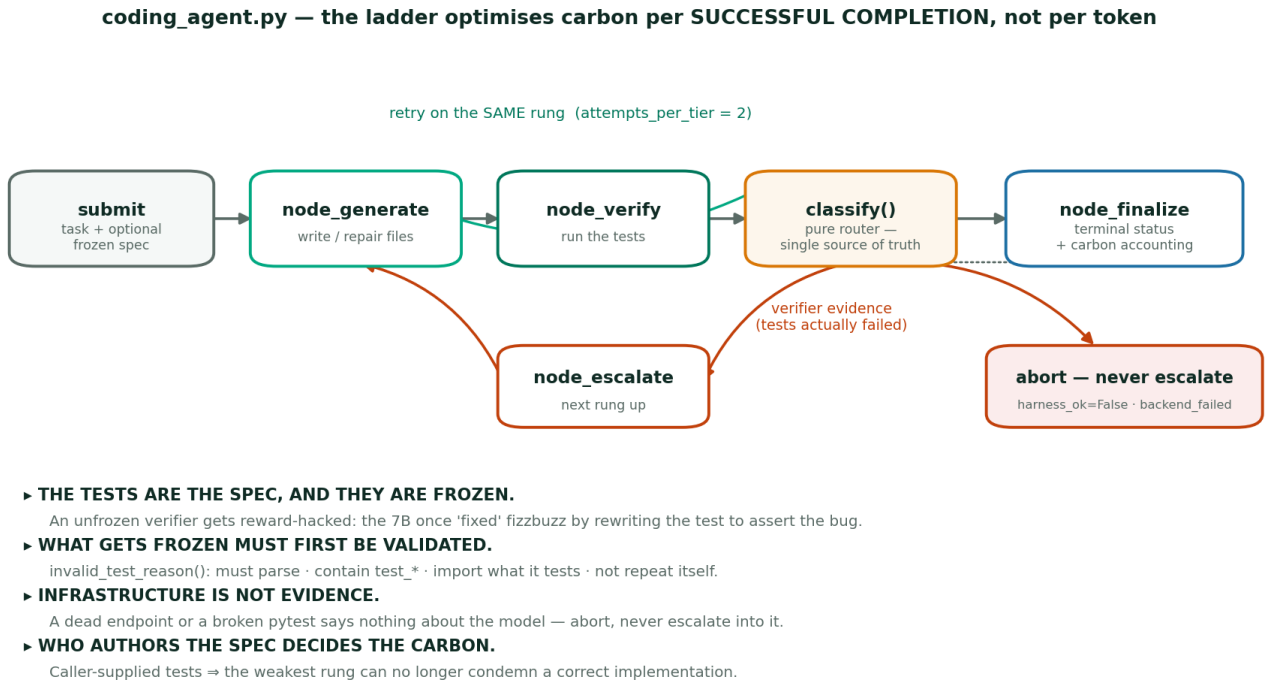
Curve shape is illustrative of a diurnal grid; the live Electricity Maps forecast is fetched at runtime (48 h @ 15 min).

A task submitted on a dirty grid is queued, not dropped, and dispatched in the greenest window the forecast offers inside its deadline. Deadline expiry still runs the task — deferral is a scheduling decision, never a silent failure.

Two implementation details are load-bearing here, and both were found by running the thing rather than by reading it. First, the queue invokes `dispatch_fn` *while holding its lock*, so a dispatch function that does minutes of work would freeze every enqueue and every status read for its whole duration; the agent's dispatch therefore spawns a worker thread and returns immediately. Second, a deferred task must re-read the grid intensity at **execution** time, not at submit time — billing it at the dirty grid it was deferred away from would erase the saving in the very books that exist to show it.

6 · The agentic coding harness

This is the newest mechanism and the one that most sharply distinguishes the system, because it is where the per-request logic *inverts*. Everything else in the platform optimises carbon per request. An agent must optimise **carbon per successful completion** — and a design that forgets the difference will confidently pick the model that guarantees the worst outcome.



The LangGraph state machine (LangGraph deliberately without LangChain). classify() is a pure function and the single source of truth for both the router and the terminal status — conditional-edge functions are routers, and state mutations inside them are silently discarded.

6.1 Escalate only on evidence

The ladder starts at Qwen2.5-Coder-1.5B — the greenest model that can actually finish a coding task — and climbs to the 7B only when the verifier says the code is wrong. The distinction that makes this safe is between **evidence** and **infrastructure**. Failing tests are evidence: the model wrote bad code, so a stronger model is worth its carbon. A missing pytest binary, a dead endpoint, an empty response from a timed-out backend — those say nothing whatsoever about the model, and escalating on them means paying 7B carbon to re-run a broken harness. A false verifier failure is the single most expensive bug this design can have, because it escalates *everything*.

```
harness_ok = False → abort. The verifier broke, not the model.
backend_failed=True → abort. An empty response is infrastructure, not evidence.
tests failed → escalate. This, and only this, is evidence.
```

6.2 The tests are the spec, and the spec is frozen

Escalation only means something if the verifier is ground truth the model cannot edit. Left unfrozen, it will not be: on the first live run, the 7B "fixed" a failing fizzbuzz by rewriting the test to assert the bug. The tests went green, the task reported `completed`, and the shipped code still returned "Fizz" for 15. That is reward hacking, and it invalidates the entire premise. So test files are immutable once written: a repair may touch the implementation and nothing else.

> But freezing something makes validating it non-negotiable — a frozen junk spec is unfixable by design.

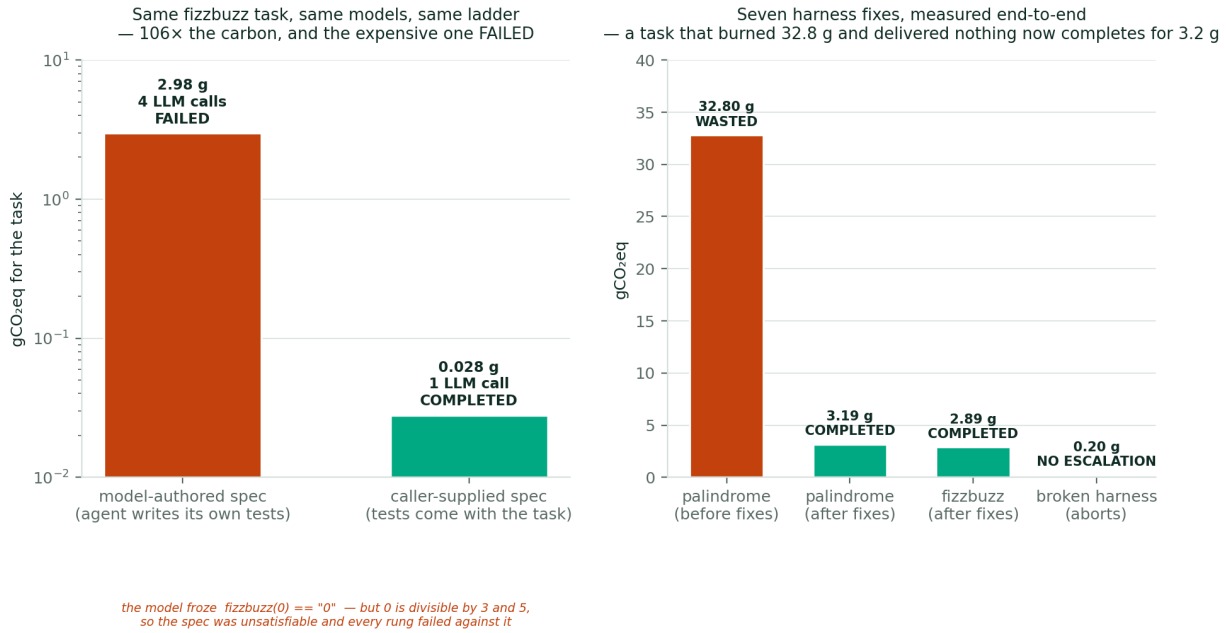
That corollary was learned the hard way too. A model echoed the prompt back inside a `test_solution.py` block; it froze; pytest could no longer collect anything; the 7B correctly tried to replace the broken file and was *refused* by the freeze — and the ladder burned its entire budget on a state it was forbidden to fix. The gate now requires that anything claiming to be a spec must parse, must contain `test_*` functions, must import what it tests (a spec that dies on NameError proves nothing), and must not repeat itself (a looping model truncates its own file at the token limit).

6.3 Carbon accounting for a loop

Field	Meaning
carbon_per_completion_g	Set only when the task actually completed. This is the metric the whole design optimises.
wasted_carbon_g	Set only when it did not. A failed agent task has no carbon-per-completion — it has pure waste, and the books say so.
deferral_ci_saved	Grid intensity at submit minus grid intensity at execution: what the deferral actually bought.
escalated / final_tier	Whether the verifier's evidence was ever strong enough to justify the bigger model.
spec_source	caller | model — who authored the ground truth this result is measured against.

6.4 Who authors the spec decides the carbon

Validation catches junk. It cannot catch a spec that is well-formed and **wrong** — and the ladder's *weakest* rung was the one authoring it. Measured on this box, twice, on different tasks: a 1.5B froze `word_count("Hello world! Hello again.") == {"hello": 2, "world": 2}` ("world" appears once; "again" is missing entirely), and — on a task as simple as fizzbuzz — it froze `fizzbuzz(0) == "0"`, when 0 is divisible by both 3 and 5 and the task statement plainly demands "FizzBuzz". Both files parse. Both import correctly. Both contain real assertions. Both are unsatisfiable, and both condemned a *correct* implementation from the rung above.



Left: the same fizzbuzz task, the same models, the same ladder — the only variable is who wrote the tests. 106× the carbon, and the expensive run **failed**. Right: the before/after of the seven harness fixes this work exposed, all measured end-to-end.

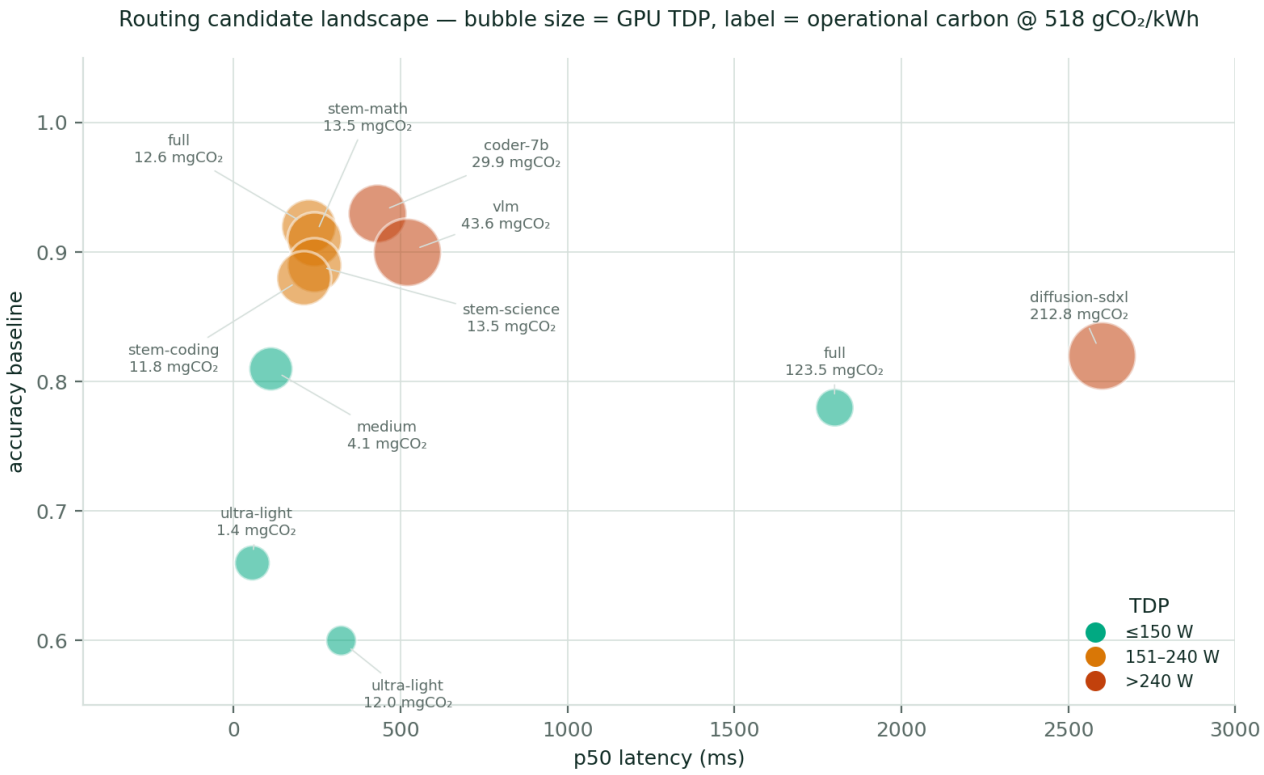
The fix is not a cleverer gate — no static analysis can know that "world" appears once in a sentence it has never seen. The fix is to stop letting the model define truth. `POST /api/agent/task` now accepts a `tests` payload: the caller's pytest suite becomes the frozen spec, validated at submit time (a bad spec is rejected with a 400 *before a single token is spent*), written into the workspace before the first model call, and the model is not permitted to emit a test file at all. Every result carries `spec_source`, because "it passed your tests" and "it passed its own tests" are very different claims and must never read the same.

7 · Low-level design

7.1 Module responsibilities

Module	Key internals
routing_policies.py	CSS scoring with min-max normalisation across the live candidate set; LLMCarbon operational + embodied terms; MoE all-to-all latency model (T_comm = k·tokens·d_model·bytes / bandwidth) that both inflates latency and degrades hardware efficiency; the semantic prompt profiler (SentenceTransformer against four prototype banks, hashed-vector fallback when the model is unavailable); the modality gate; multi-region reroute scoring.
rl_controller.py	Online REINFORCE. $\nabla \log \pi$ from a softmax over CSS scores; advantage = R − EMA baseline; decaying learning rate $\alpha_0/(1+\sqrt{t})$; simplex projection with floor w_min = 0.05; Dirichlet exploration mixed at $\varepsilon = 0.15$; convergence detected when 50 consecutive episodes hold reward variance below 0.005. Persisted to data/rl_state.json by a background saver.
advanced_rag.py	Chunk (900 / 180 overlap) → dense embedding + BM25-like sparse scoring → reciprocal-rank fusion (1/(60+rank)) → cross-encoder rerank → context cap. Ephemeral chunks let an attachment ground one request without polluting the corpus.
quality_latency_estimator.py	Online-learned per-variant corrections to the accuracy and latency inputs of CSS. Cold-start is the identity function, so it can never make a fresh deployment worse. It never touches carbon — the physics is not a learnable parameter.
deferred_queue.py	Min-heap keyed by target dispatch time; capacity 500 with explicit back-pressure (a full queue tells the caller to run inline rather than dropping the task); daemon tick every 10 s; dispatch on window-arrival or deadline-expiry, whichever comes first.
model_zoo.py	Versioned registry from config/model_zoo.json carrying the full LLMCarbon parameter set per model, plus load- and capacity-aware MoE expert placement with skew and comm-overhead estimation, and a dense fallback when placement would blow the SLA.
nemo_guardrails.py	Action-based rails, no external LLM: ~30 input patterns (jailbreak, CBRN, self-harm), a non-blocking sensitive rail, and an output rail for credential/PII leakage. Both phases land in the audit trail.
coding_agent.py	LangGraph state machine; frozen validated spec; verifier-gated escalation; per-task gCO_2 budget; sandboxed workspace with traversal-checked writes; AGENT_LLM_TIMEOUT_S = 180 s (the 45 s chat timeout throws away answers the GPU has already paid for).
conversation_store.py	SQLite in WAL mode. metadata_json carries the entire decision blob, so any message in the UI can be replayed from a single row.

7.2 The candidate registry



The decision space CSS operates over, drawn from config/model_zoo.json. Everything the router needs to reason about a candidate — accuracy, latency, TDP, PUE, hardware efficiency, manufacturing carbon, MoE topology — is declared data, not code.

7.3 Failure is designed, not discovered

Every external dependency has a defined degradation, because a sustainability platform that falls over when an API key expires is not a platform.

Dependency	When it is unavailable
Electricity Maps	Cached value, then a conservative default. The router keeps routing; it simply stops being clever about the grid.
SentenceTransformer	Hashed-vector embeddings with an identical contract. The profile gets coarser, not absent.
Cross-encoder reranker	Heuristic rerank over the fused candidates.
nvidia-smi / sidecar	Estimated GPU utilisation; /health/ready refuses to report ready until the sidecar answers.
vLLM backend	Rule-based extractive answer from retrieved evidence. For the *agent*, the same event means something different: an empty response is infrastructure, so it aborts rather than escalating.
NIM vision / diffusion	Metadata-grounded description, or an SVG placeholder. Honest degradation — the response says what it could not do.

7.4 Evidence — the HMAC-signed audit trail

Every decision becomes one signed JSONL row carrying the full semantic profile, the policy coefficients (with the RL version and whether exploration fired), the top candidate rankings with their complete CSS breakdown, the selected candidate's operational and embodied carbon, the eco-actions considered, both guardrail traces, the grid signal, the token counts, the GPU attribution and the realised latency. The signing key is server-only; any post-hoc edit invalidates the line.

> Single source of truth: /api/observability/summary keeps no state of its own. It scans the audit log. There is no metrics drift between dashboards because there is only one set of numbers.

8 · Features

Feature	Detail	Status
CSS carbon-aware routing	Four+ candidates ranked per request on carbon, latency, accuracy, cost and region, with per-tenant-tier weights.	Live
LLMCarbon accounting	Operational (TDP · PUE · hardware efficiency · live grid CI) plus amortised embodied manufacturing carbon, per candidate, per request.	Live
EcoServe deferral	48-hour forecast, ≥15 % CI-drop threshold, deadline-bounded queue with back-pressure. The coding agent is its first real caller.	Live
Online RL policy	REINFORCE with EMA baseline, Dirichlet exploration, simplex projection, convergence detection, per-tier persistence.	Live
Learned quality/latency estimator	Per-prompt corrections to the CSS accuracy and latency inputs; identity at cold start; carbon untouched.	Live
Hybrid RAG	Dense + sparse retrieval, RRF fusion, cross-encoder rerank, ephemeral per-request attachment chunks, evidence-sufficiency gating.	Live
Guardrails	Input and output rails with no external LLM dependency — the safety layer costs no carbon of its own.	Live
Semantic cache	Conversation-scoped, so a follow-up never receives a context-blind hit from another thread.	Live
Agentic coding harness	LangGraph ladder, verifier-gated escalation, validated frozen spec, caller-supplied tests, per-task carbon budget, EcoServe-deferrable.	Live
Multimodal routing	Modality gate (text / vision / image-gen); carbon-capped diffusion steps above a CI threshold; deferrable generation.	Live — dispatch
Vision pixel analysis	Requires a VLM endpoint (NIM_VLM_URL, or any OpenAI-compatible vision server). Unset ⇒ metadata-grounded fallback, no pixels read.	Needs endpoint
Observability	KPIs, period-over-period deltas, SLO + error budget, latency heatmap, per-model rollup, anomaly z-scores, trace explorer with CSV export.	Live
Signed audit trail	HMAC-SHA256 per decision; the single source of truth behind every dashboard.	Live
Feedback capture	Thumbs up/down per assistant message → SQLite → JSONL export, seeding offline LoRA fine-tuning.	Live
CSRD reporting	Period energy/carbon rollup with market-based renewable accounting; CSV export.	Live

8.1 Honest limits

A document that lists only what works is marketing. These are the edges as they stand today.

Limit	Consequence, and the fix
No pixel-level vision without an endpoint	Image attachments are routed correctly to the vision path, but with no VLM configured the response is grounded in metadata (filename, size, dimensions), not in the image itself. Fix: point NIM_VLM_URL at a NIM or any OpenAI-compatible vision server — the dispatch code already sends image_url content and needs no change.
Model-authored specs can be wrong	When the caller supplies no tests, the weakest rung still authors the frozen spec, and a confidently wrong one condemns a correct implementation (§6.4). Mitigation shipped: caller-supplied tests. The default path remains exposed by design — the alternative is refusing to run without a spec.
The task registry is in-memory	A restart loses queued agent tasks. So does the deferred queue itself, so persisting one without the other would only produce records of tasks nothing will ever run. The audit log remains the durable record.
No automated test suite	Verification is end-to-end against the running stack. Every number in this document came from an actual run, which is a strength for the claims and a weakness for regression safety.

9 · Why the design holds together

Principle	How it is enforced
Carbon is an input, not a report	The grid signal participates in CSS scoring, in SLA penalties, in the MoE go/no-go, in the deferral decision, in the diffusion step budget, and in the RL reward. Remove the dashboard and the system still makes greener decisions; remove the grid signal and it demonstrably makes worse ones.
One source of truth	Every dashboard, every KPI and every replay derives from data/decision_logs.jsonl. No component keeps its own counters, so no two views can disagree.
Evidence, never inference, drives escalation	The agent climbs the ladder only when the verifier proves the code is wrong. Infrastructure failures abort. This one rule is what keeps carbon-per-completion bounded.
What you freeze, you must first validate	An immutable verifier is the only thing standing between the harness and reward hacking — and an immutable *junk* verifier is unfixable by construction. Both halves are enforced at the gate.
Graceful degradation everywhere	Every external dependency has a defined fallback with an identical contract. The system gets less clever, never unavailable.
Learning is free	The RL update runs off the request path, so the platform can afford to learn from every single request instead of sampling.

> The measured result: an agentic coding task that burned 32.8 gCO_2 and delivered nothing now completes for 3.2 g — and with a caller-supplied spec, for 0.03 g on the greenest rung, without ever waking the larger model.

Every figure in this document was produced by running the system described in it: the carbon numbers come from config/model_zoo.json through the LLMCarbon formula the router itself uses, the RL curve comes from data/rl_state.json on this deployment, and the agent measurements come from actual POST /api/agent/task runs on 13 July 2026. Where a chart is illustrative rather than measured, it says so on the chart.